

Principios y patrones de diseño de software

Contenidos

- Principios y patrones de diseño de software
 - Software de buena calidad
 - Cumplir las expectativas del usuario
 - Utilizar diseño orientado a objetos
 - Utilizar principios y patrones de diseño de software
 - Elegir buenos nombres para clases, propiedades y métodos
 - Principios de diseño de software
 - Principio Abierto–Cerrado
 - Técnica de programación: Refactorizar
 - Principio No te Repitas
 - Principio de Sustitución de Liskov
 - Patrones de diseño de software
 - Unico en su clase, el patrón Singleton
 - Aprende a delegar, el patrón estrategia (strategy)
 - Retorno al pasado, el patrón orden (command)
 - Mini ejercicios

Software de buena calidad

En las sesiones anteriores hemos aprendido a programar en Dart. Ahora es el momento de poder escribir **buenas aplicaciones** usando este lenguaje. Pero antes conviene reflexionar sobre:

¿Qué entendemos por **software de buena calidad**?

Un buen software debe:

1. Hacer lo que el usuario necesita que haga.
2. Aplicando diseño orientado a objetos para añadir máxima flexibilidad.
3. Con un diseño reutilizable y fácil de cambiar.

Parece sencillo, pero en realidad no lo es. Veamos cada uno de esos puntos con un poco más de detalle.

Cumplir las expectativas del usuario

Un gran software debe satisfacer a sus usuarios. El software debe hacer lo que los usuarios quieren que haga. Para ello hay que tener unos buenos requisitos bien probados con los usuarios y no solo una vaga idea de lo que queremos que haga la aplicación.

Si conseguimos esto, nuestros usuarios estarán muy satisfechos con la aplicación, la usarán mucho y la recomendarán a otros usuarios. Eso es muy bueno ya que aumenta nuestro volumen de negocio.

¿Esto por sí solo es suficiente para considerar que un programa es software de calidad?

Atención spoiler: No.

Utilizar diseño orientado a objetos

Un gran software debe estar orientado a objetos. De esta forma no hay un montón de código duplicado, cada clase encapsula una parte del comportamiento de la aplicación y cada objeto controla sus propios datos y su funcionamiento.

Todo eso hace que el código sea más sencillo de escribir y entender por el resto de miembros del equipo de desarrollo. También hace que el software tenga un diseño sólido y fiable.

¿Las dos características anteriores son suficientes para considerar que un programa es software de calidad?

Atención spoiler: Siento repetirme pero todavía no.

Utilizar principios y patrones de diseño de software

¿Qué pasa cuando es el momento de ampliar el software con nuevas funciones para lanzar una nueva versión? ¿o qué pasa cuando se necesita reutilizarlo en otra aplicación?

No basta con tener un software orientado a objetos que funcione como el usuario quiere; el software debe adaptarse bien a los **cambios**.

"Nada es permanente a excepción del cambio"

Heráclito

Todos los programas están en continuo cambio. Continuamente hay que lanzar nuevas versiones con más y mejores funcionalidades para satisfacer nuevos requisitos del usuario o para superar a los programas de la competencia.

Muy importante: La única cosa que **siempre** va a ocurrir durante la vida de cualquier proyecto software es el:

CAMBIO

No importa lo bien diseñada que esté una aplicación, con el tiempo deberá **crecer y cambiar o morirá**.

Si se utilizan **principios y patrones de diseño** bien probados, hacer cambios será una tarea sencilla, ya que los cambios afectarán a una parte pequeña del software y no afectarán para nada al resto. Eso también ayuda a que el código sea más reutilizable, por lo que no habrá que reescribir todo de nuevo para usar partes de una aplicación en un nuevo proyecto.

No parece que sea una tarea sencilla.

Atención spoiler: No lo es, sin embargo los principios y patrones de diseño que se estudian en este bloque harán que tu software cambie a mejor para siempre.

Elegir buenos nombres para clases, propiedades y métodos

Antes de comenzar a estudiar los principios y patrones de diseño, vamos a detenernos en un elemento clave a la hora de tener un buen diseño orientado a objetos: el nombrado correcto de los componentes.

Siempre se deben usar nombres de métodos que definan claramente lo que hacen. Esta técnica sencilla ahorra muchas líneas de comentarios y ayuda a entender lo que hace un programa.

A la hora de escribir buenos nombres de métodos, los parámetros con nombre pueden ayudar. Por ejemplo:

```
void imprimirEstudiante({required String nombre, required String id})
```

es mucho mejor que:

```
void imprimir(String nombre, String id)
```

ya que a la hora de llamar a esos métodos se ve claramente las ventajas del primero sobre el segundo.

```
objeto.imprimirEstudiante(nombre: 'Francisco', id: 'AJ23-K');  
objeto.imprimir('Francisco', 'AJ23-K');
```

Las ventajas son todavía mayores cuando entre los parámetros de un método se encuentran varios parámetros del mismo tipo. Por ejemplo, el constructor:

```
var circulo = Circulo(radio: 10.0, grosor: 2.0);
```

es mucho más claro que:

```
var circulo = Circulo(10.0, 2.0);
```

En este último ¿qué significan esos argumentos? ¿cuál corresponde con el radio y cuál con el grosor?

Importante: También debemos dar a las propiedades y a las clases, nombres que expliquen por sí solos lo que contienen o cuál es su responsabilidad. Por ejemplo:

```
numeroDePersonas  
mediaPonderada  
fechaSuscripcion
```

y en el caso de clases:

```
Usuario  
RepositorioSimulado  
Spaceship
```

Principios de diseño de software

Un principio de diseño de software es una herramienta o técnica básica que se puede aplicar al diseño o escritura de código para que ese código sea más mantenible, flexible o extensible.

Puede que te estés preguntando:

¿Por qué debería estudiar principios de diseño de software?

No hay nada más satisfactorio que llegar a una solución nueva y original a un problema que te ha estado preocupando durante días, hasta que descubres que alguien ya había resuelto el mismo problema, mucho antes que tú, e hizo un trabajo aún mejor que tú.

En este apartado, vamos a ver algunos principios de diseño de software que otros ingenieros han ideado a lo largo de los años, y cómo pueden hacer de ti un mejor programador. Deja a un lado la idea de *"hacerlo a tu manera"*; este apartado trata sobre hacerlo de **la manera más inteligente y rápida**.

"La originalidad está muy sobrevalorada. Para hacer, necesitas tomar."

Darby Bannard

Principio Abierto–Cerrado

El primer principio de diseño de software que vamos a estudiar es el **principio abierto–Cerrado**, **OCP** (Open–Close principle). El OCP consiste en permitir el cambio, pero hacerlo sin requerir que se modifique el código existente.

Definición del principio abierto–cerrado

Las clases deben estar abiertas para su extensión pero cerradas para su modificación.

¿Recuerdas el ejemplo del tipo enumerado `ColorSemaforo` que se estudió en el apartado dedicado a los tipos enumerados mejorados del bloque anterior? Es una aplicación clara del principio abierto–cerrado.

Se tenía la siguiente declaración del tipo enumerado:

```
enum ColorSemaforo {  
    verde,  
    amarillo,  
    rojo,  
}
```

y cuando se quería imprimir por la consola un mensaje en función de la constante del tipo enumerado que se tuviera, había que recurrir a un `switch` :

```
final color = ColorSemaforo.amarillo;  
switch(color) {  
    case ColorSemaforo.verde:  
        print('Avanzar');  
        break;  
    case ColorSemaforo.amarillo:  
        print('Decelerar');  
        break;  
    case ColorSemaforo.rojo:  
        print('Detenerse');  
        break;  
}
```

El fragmento de código anterior imprime en la consola:

```
Decelerar
```

El problema de este código es que **no es simple de extender** porque **incumple el principio abierto–cerrado**.

Si en una versión futura de la aplicación se viese necesario añadir un cuarto color, por ejemplo `azul` que tuviese asociado el mensaje `Apartarse a un lado`, habría que hacer dos cosas:

- **Añadir** una nueva constante `azul` al tipo enumerado `ColorSemaforo`
- **Modificar** el fragmento de código que lo usa añadiendo un nuevo `case` al `switch` con el nuevo mensaje.

La primera de ellas no es problemática, se está añadiendo una nueva funcionalidad y para ello hay que añadir nuevas cosas al código.

La segunda es la que causa problemas. No puede ser que para poder usar esa nueva funcionalidad haya que **modificar** el código que usa la clase (el tipo enumerado en este ejemplo).

Técnica de programación: Refactorizar

Cuando descubramos que nuestro código no está estructurado de una manera conveniente para añadirle nueva funcionalidad. Antes de comenzar a añadir la nueva funcionalidad hay que **refactorizarlo** para que sea fácil añadirla. Solo entonces la añadiremos. Cualquier otro enfoque nos traerá problemas en el futuro.

Importante: La técnica de refactorización es muy importante ya que ninguno de nosotros, por mucha experiencia que tengamos, vamos a diseñar código 100% adaptado a todos los cambios futuros. Siempre vamos a descubrir pequeñas cosas que hacen que las nuevas funcionalidades no se integren del todo bien con las actuales.

Es una técnica cuyo estudio profundo se sale del ámbito de una asignatura de desarrollo de aplicaciones móviles, pero en esta sección voy a dar unas pinceladas sobre cómo funciona.

Definición de refactorización

Cambio realizado en la estructura interna del software para que sea más fácil de entender y más sencillo de modificar **sin cambiar su comportamiento**.

Lo importante de la refactorización es que se cambia la estructura interna del código, pero no se cambia su funcionalidad o comportamiento. Se trata de que siga haciendo lo mismo, pero estructurado de forma que las nuevas funcionalidades se integren con las actuales sin que haya que modificar estas últimas.

Muy importante: La forma correcta de refactorizar un programa es cambiarlo **en pequeños pasos**. Nunca hay que intentar cambiarlo todo de golpe. Así, si se comete un error, es fácil encontrar dónde está.

En este ejemplo concreto, usaremos la capacidad de Dart de tratar a los tipos enumerados como clases y encapsularemos el mensaje asociado a cada constante del tipo junto a esa constante.

1. Comenzaremos añadiendo una propiedad al tipo enumerado de clase `String` a la que llamaremos `mensaje`

```
final String mensaje;
```

Las propiedades de un tipo enumerado deben definirse como `final` ya que los elementos del tipo son constantes y no tiene ningún sentido que sus propiedades puedan cambiarse una vez creadas.

Si no se declara como `final` se obtiene el error de compilación:

```
Enums can only declare final fields
```

2. A continuación hay que añadir un constructor para poder inicializar esa propiedad.

```
const ColorSemaforo(this.mensaje);
```

El constructor debe declararse como `const` De lo contrario se produce el error de compilación:

```
Error: Generative enum constructors must be marked as 'const'.
```

Es lógico, ya que las *instancias* de un tipo enumerado son las constantes del tipo.

3. Cuando en un tipo enumerado hay propiedades, las constantes se declaran mediante el constructor. Por tanto hay que cambiar el código actual para que llame al constructor al construir cada constante. Cambiaremos:

```
verde,  
amarillo,  
rojo,
```

por:

```
verde('Avanzar'),  
amarillo('Decelerar'),  
rojo('Detenerse');
```

Ojo con el `;` final. Si no se escribe nada o se escribe una coma `,` se obtiene el error de compilación:

```
Error: Expected '}' before this.
```

El tipo enumerado refactorizado queda así:

```
enum ColorSemaforo {  
  verde('Avanzar'),  
  amarillo('Decelerar'),  
  rojo('Detenerse');  
  
  final String mensaje;  
  const ColorSemaforo(this.mensaje);  
}
```

Ahora hay que refactorizar el uso que se hace del tipo enumerado. Para ello se sustituye el `switch` original por:

```
final color = ColorSemaforo.amarillo;  
print(color.mensaje);
```

Ya hemos terminado la refactorización. Ahora nuestro código está preparado para añadir la nueva constante sin tener que modificar nada mas. Para ello sustituimos:

```
rojo('Detenerse');
```

por:

```
rojo('Detenerse'),  
azul('Apartarse a un lado');
```

Como vemos, añadir el nuevo color al semáforo, sólo ha requerido **añadir código** pero no **modificar** el código usuario existente.

Con el mismo código de uso:

```
print(color.mensaje);
```

si ahora la variable `color` tiene asignado el valor `ColorSemaforo.azul`, el resultado que se obtiene en la consola es:

```
Apartarse a un lado
```

Nota: Este tipo de técnicas se usan mucho cuando hay que rellenar las vistas de una interfaz de usuario con elementos que pueden variar en el futuro, por ejemplo, los casos de un desplegable o Combo Box. Si se programan correctamente, se añadirán los nuevos casos y automáticamente la interfaz los incorporará al Combo Box, sin tener que hacerlo manualmente.

Lo veremos más adelante, cuando hagamos aplicaciones móviles con Flutter.

Principio No te Repitas

El siguiente principio que vamos a estudiar es el **principio No te repitas**, **DRY** (Don't Repeat Yourself). Este es otro principio que parece bastante simple, pero que es muy útil para escribir código fácil de mantener y reutilizar.

Definición del principio no te repitas

Hay que evitar duplicar el código abstrayendo las cosas que son comunes y colocándolas en un solo lugar.

Abstraer el código duplicado es un buen comienzo para usar el principio no te repitas, pero va mucho más allá. Cuando intentas evitar duplicar el código, realmente lo que estás haciendo es implementando cada necesidad de tu aplicación en un único lugar.

Importante: DRY va de tener cada pieza de **información** y **comportamiento** de tu aplicación en un único lugar.

En dart se pueden usar propiedades y métodos para implementar el principio no te repitas.

1. Para evitar repetir un determinado valor en diferentes partes del código, se puede encapsular en una propiedad, que puede ser de instancia o de clase, y usar esa propiedad para acceder al valor.

Esto ayuda a la hora de hacer cambios, ya que solo habrá que cambiar el valor que se asigne a la propiedad y no el valor en los múltiples sitios donde pueda estar usándose.

2. Para evitar repetir una determinada funcionalidad en diferentes partes del código, se puede encapsular un fragmento de programa en una función o método, que puede ser de instancia o de clase, y usar esa función o método siempre que se necesite ejecutar esa funcionalidad.

Esto ayuda a la hora de hacer cambios, ya que si se decide cambiar la forma en que se implementa la funcionalidad, solo habrá que modificar el código en la función o método donde se implementa y no en los múltiples sitios donde pueda estar usándose.

En la sesión de prácticas siguiente se propone refactorizar un código para que no se viole el principio no te repitas, DRY. Por eso, en esta sección no se va a dar ningún ejemplo concreto.

Principio de Sustitución de Liskov

El último principio que vamos a estudiar en esta sesión es el principio de sustitución de Liskov, LSP (Liskov Substitution Principle). Es un principio fundamental, ya que nos permite comprobar si una jerarquía de clases está bien diseñada, es decir, si estamos usando bien la especialización (herencia).

Definición del principio de sustitución de Liskov

Las subclases deben ser sustituibles por sus superclases.

El principio de sustitución de Liskov, LSP, permite comprobar si una jerarquía de clases está bien diseñada. Cuando se especializa una clase base, debes ser capaz de sustituir tu subclase por esa clase base sin que las cosas vayan terriblemente mal. De lo contrario, ¡hemos usado la herencia incorrectamente!

A menudo se habla de la especialización como **herencia** y se piensa que es una gran ventaja poder heredar los métodos de una clase en otra que tiene **comportamiento parecido**. De esta forma se ahorra tiempo al no tener que codificar todo desde el principio ya que como el comportamiento es parecido, entonces se puede reutilizar en gran parte. Eso es incorrecto y lleva con frecuencia a un uso desastroso de la especialización.

Un momento: Hasta ahora hemos dicho que la especialización es un mecanismo muy potente y que es fundamental en el diseño orientado a objetos. ¿Cómo es posible que una jerarquía de clases tenga consecuencias desastrosas?

Como todas las cosas mal diseñadas, su uso es complejo y no solo no facilita las cosas, sino que las empeora.

La clave aquí es **comportamiento parecido**. Eso es un problema, ya que la relación de especialización es una relación **ES UN** y no **PARECIDO A** o **SIMILAR A**.

Muy importante: En una relación de especialización, las subclases **SON** como las superclases en el sentido de que se puede usar una o la otra sin tener que modificar la forma en que se usan. De esta forma como todas las clases en la jerarquía son intercambiables, todas funcionan de la misma forma, **el código aguanta muy bien los cambios**.

Para construir una jerarquía de clases que respete el principio de sustitución de Liskov hay que asegurarse de que cuando se extiende una clase **no se está alterando el comportamiento de la superclase**. La subclase es una versión especializada de la superclase, pero no se ha convertido en otra cosa parecida pero no igual.

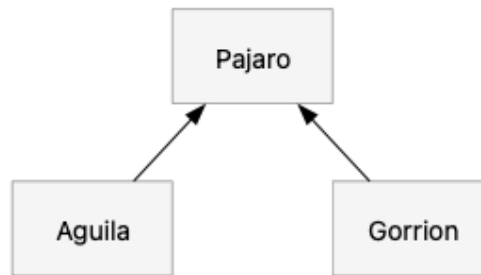
Veamos un ejemplo sencillo.

Nota: La solución que se propone aquí no es la mejor, pero sirve perfectamente para ilustrar cual es el problema cuando se viola el principio LSP.

Imagina un juego que dibuja en la pantalla pajaritos a diferentes alturas según su tipo. En la primera versión solo hay águilas y gorriones. Cada una de ellas tiene un rango de altura de vuelo, y dentro de ese rango, cada instancia vuela a una altura concreta aleatoria.

Podríamos pensar en una jerarquía de clases en la que la superclase sea la clase **Pajaro** y tenga dos subclases: **Aguila** y **Gorrion**.

La jerarquía de clases se vería así:



y en Dart podría implementarse así:

1. Se define la superclase `Pajaro`

```
class Pajaro {
  var altitudActual = 0;

  void iniciaVuelo() {
    var rnd = Random();
    altitudActual = rnd.nextInt(300);
  }
  @override
  String toString() {
    return 'Pájaro. Altura actual: $altitudActual';
  }
}
```

- La clase `Pajaro` tiene una propiedad `altitudActual` que indica la altura a la que está volando el pájaro en este momento.
 - Define un método `iniciaVuelo()` que inicializa el valor de `altitudActual` de forma aleatoria, dentro del rango de valores definido. En este caso: `0..300`
 - Se ha redeclarado el método `toString` para simular el proceso de dibujo del pájaro en un canvas. Lo que hace es imprimir un mensaje por consola indicando el tipo de pájaro y su altitud actual.
2. Lo siguiente es declarar una subclase `Gorrion` para implementar uno de los tipos de pájaros concretos de la aplicación.

```
class Gorrion extends Pajaro {
  @override
  void iniciaVuelo() {
    var rnd = Random();
    altitudActual = rnd.nextInt(100);
  }
  @override
  String toString() {
    return 'Gorrión. Altura actual: $altitudActual';
  }
}
```

- El gorrión vuela más bajo que el resto de pájaros, por tanto se ha redefinido el método `iniciaVuelo()` usando otro rango de altura.

- También se ha redeclarado el método `toString` para que imprima el mensaje adecuado para el tipo de pájaro `Gorrion`

3. El otro tipo de pájaro que se requiere en la versión 1.0 del juego es el águila. Para implementarlo en este esquema hay que definir una clase `Aguila` que sea subclase de `Pajaro`

```
class Aguila extends Pajaro {  
    @override  
    String toString() {  
        return 'Aguila. Altura actual: $altitudActual';  
    }  
}
```

- Las águilas vuelan a la altura de vuelo normal definida en `Pajaro` por tanto hereda el método `iniciaVuelo()` sin necesidad de hacerle cambios.
- Se redeclara el método `toString` para que en la pantalla aparezca el mensaje correcto.

4. Para poder usar esa jerarquía de clases hace falta una clase `Canvas` que simula la pantalla donde se dibujan los pájaros en el dispositivo móvil. Podría declararse así:

```
class Canvas {  
    List<Pajaro> pajaros = [];  
  
    Canvas(this.pajaros);  
  
    void dibuja() {  
        for (final pajaro in pajaros) {  
            pajaro.iniciaVuelo();  
            print('Dibujando $pajaro');  
        }  
    }  
}
```

- La clase `Canvas` tiene como propiedad una lista de `Pajaro` llamada `pajaros` inicializada con una lista vacía.
- En el constructor se pasa la lista de pájaros que haya que utilizar en el juego.
- Por último, dispone de un método `dibuja` que se llamará cada vez que se tenga que refrescar la interfaz de usuario. En este ejemplo, lo que hace es iterar sobre la lista de pájaros, lanzarlos a volar e imprimir por la consola su información.

5. Un uso del `Canvas` en el `main` de la aplicación puede ser:

```
var canvas = Canvas([Aguila(), Gorrion()]);  
canvas.dibuja();
```

que muestra en la consola:

```
Dibujando Aguila. Altura actual: 249  
Dibujando Gorrion. Altura actual: 71
```

Como podemos ver, funciona bien. Ahora es el momento de ver qué tal soporta los cambios. En la versión 2.0 nos piden añadir un nuevo pájaro, el `Pingüino`. Se declara como una subclase de `Pajaro` y comienzan nuestros problemas.

```
class Pinguino extends Pajaro {  
    @override  
    String toString() {  
        return 'Pingüino. Altura actual: $altitudActual';  
    }  
}
```

Los pingüinos no vuelan, por tanto el método `iniciaVuelo()` que hereda de `Pajaro` y que calcula una altura de vuelo aleatoria dentro de un rango, no tiene sentido. Los pingüinos no inician el vuelo, porque no vuelan.

Podría redeclararse el método `iniciaVuelo()` en la clase `Pinguino` y hacerlo vacío. No es una solución muy robusta.

Otra opción es redeclararlo para que lance un error para asegurar que nadie invoque el método `iniciaVuelo()` de la clase `Pinguino`

```
@override  
void iniciaVuelo() {  
    print('Los pingüinos no vuelan. No use este método');  
    exit(1);  
}
```

Se elija la opción que se elija, se viola el principio de sustitución de Liskov, ya que el programa no puede utilizar cualquier subclase de `Pajaro` en el `Canvas` y seguir siendo válido.

El siguiente fragmento de código en el `main`:

```
var canvas = Canvas([Aguila(), Gorrión(), Pinguino()]);  
canvas.dibuja();
```

produce la siguiente salida:

```
Dibujando Aguila. Altura actual: 243  
Dibujando Gorrión. Altura actual: 50  
Los pingüinos no vuelan. No use este método  
  
Process finished with exit code 1
```

y la terminación de la aplicación. No se puede dejar la aplicación así.

Una posible solución es reescribir el método `dibuja()` de la clase `Canvas` para que distinga la situación en la que un elemento de la lista es un `Pinguino` y sobre él no aplique el método `iniciaVuelo()`

```
void dibuja() {  
    for (final pajaro in pajaros) {  
        if (pajaro is! Pinguino) {  
            pajaro.iniciaVuelo();  
        }  
        print('Dibujando $pajaro');  
    }  
}
```

- Se usa `is!` para comprobar si el elemento que se está procesando no es de clase `Pinguino`. Si no lo es se le aplica el método `iniciaVuelo()` y si lo es no.

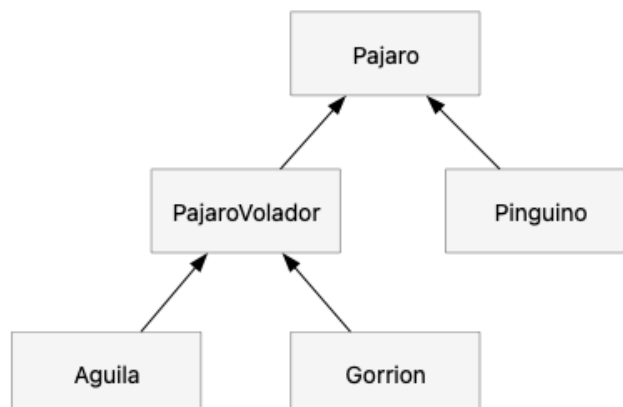
Hemos solucionado el problema. Ya no se viola el principio de sustitución de Liskov, pero ahora se viola el principio abierto-cerrado. Para añadir un nuevo pájaro, el pingüino, se ha tenido que modificar el comportamiento de la clase usuario `Canvas`. Eso es un error.

Además, si se añaden a la aplicación nuevos pájaros que no vuelan. Por ejemplo un `Pollo`, hay que rehacer el `if` para añadirle la comprobación `pajaro is! Pollo` lo que lo hace más confuso de leer y editar.

La solución definitiva pasa por darse cuenta de que la jerarquía de clases que se ha definido no es correcta y rehacerla para que lo sea.

Importante: La norma principal que se debe seguir al definir una jerarquía de clases, es que todo lo que la superclase pueda hacer, lo deben poder hacer las subclases. En este ejemplo, en la superclase se tiene la capacidad de iniciar el vuelo mediante el método `iniciaVuelo()`, así que todas sus subclases deben tener también esa capacidad.

Si se sigue esa norma, tendremos una jerarquía de clases con una clase nueva `PajaroVolador` que es subclase de `Pajaro`. En ella es donde se define el método `iniciaVuelo()` y no en `Pajaro` y los pájaros no voladores como el `Pollo` y el `Pinguino` son subclases de `Pajaro` que ya no tiene el método `iniciaVuelo()`.



1. La clase `Pajaro` queda así:

```
class Pajaro {
    var altitudActual = 0;

    @override
    String toString() {
        return 'Pájaro. Altura actual: $altitudActual';
    }
}
```

- Se ha eliminado el método `iniciaVuelo()` ya que no tiene sentido en todos los pájaros.

2. La clase `Pinguino` queda así:

```
class Pinguino extends Pajaro {
    @override
    String toString() {
        return 'Pingüino. Altura actual: $altitudActual';
    }
}
```

- Sigue extendiendo de la clase `Pajaro`, pero como ahora ésta no tiene el método `iniciaVuelo()` puede sustituirse por la superclase sin ningún problema.

3. Se ha añadido una nueva clase a la jerarquía `PajaroVolador` así:

```
class PajaroVolador extends Pajaro {
    void iniciaVuelo() {
        var rnd = Random();
        altitudActual = rnd.nextInt(300);
    }
}
```

- Es una subclase de `Pajaro`
- El método `iniciaVuelo()` se ha movido a esta clase, de forma que sus subclases lo tendrán disponible y lo podrán heredar o redeclarar si lo necesitan.

4. Los pájaros voladores `Gorrion` y `Aguila` se han hecho subclases de `PajaroVolador`

```
class Gorrion extends PajaroVolador {
    @override
    void iniciaVuelo() {
        var rnd = Random();
        altitudActual = rnd.nextInt(100);
    }
    @override
    String toString() {
        return 'Gorrión. Altura actual: $altitudActual';
    }
}
```

- El gorrión vuela en un rango de altura mas bajo que el declarado en la superclase, por tanto debe redeclarar el método `iniciaVuelo()` .
- También debe redeclarar el método `toString`

```
class Aguila extends PajaroVolador {
    @override
    String toString() {
        return 'Aguila. Altura actual: $altitudActual';
    }
}
```

- El águila vuela dentro del rango de altura definido en `PajaroVolador` por tanto solo tiene que redeclarar el método `toString`

5. La clase usuaria `Canvas` debe reescribirse así:

```
class Canvas {
    List<Pajaro> pajaros = [];

    Canvas(this.pajaros);

    void dibuja() {
        for (final pajaro in pajaros) {
            if (pajaro is PajaroVolador) {
                pajaro.iniciaVuelo();
            }
            print('Dibujando $pajaro');
        }
    }
}
```

Con todos esos cambios, si se ejecuta el siguiente fragmento de código en la función `main` :

```
var canvas = Canvas([Aguila(), Gorrión(), Pinguino()]);
canvas.dibuja();
```

se obtiene la siguiente salida en la consola:

```
Dibujando Aguila. Altura actual: 86
Dibujando Gorrión. Altura actual: 26
Dibujando Pingüino. Altura actual: 0
```

Muy importante: Detectar si se está violando el principio de sustitución de Liskov es más sencillo de lo que parece. Los siguientes síntomas indican que se está violando el LSP:

- Si se están creando subclases de una superclase y se nota que a las subclases **les sobra algún método de la superclase** que acaba dejándose vacío o lanzando alguna excepción, entonces es muy probable que se esté violando el LSP.
- Si para usar una subclase primero hay que averiguar qué tipo tiene (a qué subclase pertenece) y en función de ello ejecutar un código u otro, también es probable que se esté violando este principio.

Nota: Todo lo anterior nos ha servido para ilustrar los problemas de no usar una jerarquía de clases bien diseñada, pero **la solución propuesta no es la óptima**. Hay una solución mejor que pasa por usar el **patrón estrategia** que se estudia un poco más adelante.

Patrones de diseño de software

Los **patrones de diseño de software** son plantillas reutilizables de soluciones para problemas comunes de desarrollo de software. Los patrones de diseño no son implementaciones concretas, sino que sirven como puntos de partida para escribir código. Describen soluciones genéricas y **bien probadas** para problemas que los desarrolladores de software han encontrado a lo largo de su trabajo.

El uso de patrones de diseño ahorra mucho tiempo de desarrollo, ya que evita tener que dedicar tiempo a idear una solución para un problema y garantiza que la solución que aporta el patrón funciona correctamente y produce software capaz de adaptarse a los cambios de forma sencilla.

Unico en su clase, el patrón Singleton

El patrón singleton es la solución al siguiente problema: se necesita una clase:

- de la que solo se pueda construir **una única instancia**.
- que sea **accesible** desde cualquier otra clase o función de la aplicación, **sin que haya acoplamiento**.

Veamos el patrón con un ejemplo sencillo:

```
class Singleton {
    // Un singleton, como cualquier otra clase, puede tener tantas propiedades
    // como sea necesario. En este ejemplo se ha declarado estas dos.
    int unaPropiedad = 10;
    String otraPropiedad = 'Prueba singleton';

    // La única instancia que puede construirse de esta clase. Esta propiedad está
    // marcada como final, para que no pueda ser cambiada una vez inicializada, y
    // es una propiedad de clase (static) para que pueda ser accedida, a través de la
    // clase, desde cualquier otra clase o función de la aplicación
    static final Singleton unicaInstancia = Singleton._();

    // Nótese que este constructor es privado. solo puede usarse desde el archivo
    // en el que se declara la clase.
    // De esta forma no pueden construirse nuevas instancias desde fuera.
    Singleton._();

    // Como en cualquier otra clase se pueden definir tantos métodos como sea
    // necesario, por ejemplo: toString para poder imprimirlo fácilmente.
    @override
    String toString() {
        return 'Singleton(unaProp: $unaPropiedad, otraProp: $otraPropiedad)';
    }
}
```

Como vemos no es difícil aplicar el patrón. Los puntos claves son:

- **Definir el constructor como privado**, ya que de esta forma se garantiza que nadie puede crear nuevas instancias desde fuera de la clase.
- Definir, dentro de la clase, **una propiedad de clase** (marcada como `static`) cuyo tipo sea la propia clase. Esa propiedad es una referencia a la **única instancia** que se permite de esa clase. El hecho de que sea de clase, permite su acceso desde cualquier otra clase o función, sin ningún tipo de acoplamiento.

Para usar la clase `Singleton` se accede a la propiedad `unicaInstancia` a través de la clase `Singleton`.

Posibles usos son:

```
void main() {  
    // Para acceder al singleton, se llama a la propiedad de clase 'unicaInstancia'  
    // Al ser de clase, puede llamarse desde cualquier módulo sin que esté  
    // acoplado con el singleton.  
    print(Singleton.unicaInstancia);  
  
    Singleton.unicaInstancia.unaPropiedad = 4;  
    Singleton.unicaInstancia.otraPropiedad = 'Cambiando';  
    print(Singleton.unicaInstancia);  
}
```

Si se ejecuta el fragmento de código anterior, se obtiene la siguiente salida en la consola de depuración:

```
Singleton(unaProp: 10, otraProp: Prueba singleton)  
Singleton(unaProp: 4, otraProp: Cambiando)
```

Aprende a delegar, el patrón estrategia (strategy)

El patrón estrategia es la solución al siguiente problema:

- Definir una **familia de algoritmos, encapsulados e intercambiables**. De esta forma cada algoritmo puede variar independientemente de los clientes que lo usan.

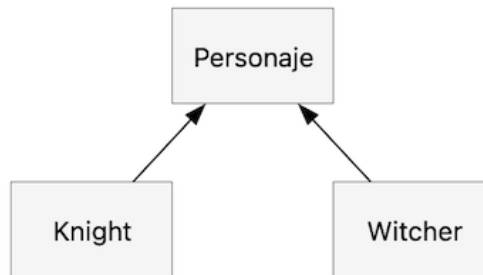
Veamos un ejemplo sencillo:

Se está desarrollando un juego de aventura en el que hay personajes de dos tipos: **Knight** (caballero) y **Witcher** (bruja). Los caballeros tienen una forma de ataque llamada: **super powerful attack** y las brujas un: **magic attack**. En momentos concretos del juego los personajes pueden evolucionar y cambiar de rol. Por ejemplo, un caballero podría convertirse en un brujo.

Una forma de implementar esto es mediante una relación de especialización. Se define una superclase `Personaje` con una propiedad `nombre` de tipo `String` y un método `atacar()` virtual puro.

A continuación se especializan dos subclases de `Personaje`: `Knight` y `Witcher` donde se redefine el método `atacar()` con las peculiaridades de cada subclase concreta.

El diagrama de clases quedaría así:



y la implementación en Dart podría ser así:

En primer lugar la superclase `Personaje`:

```
abstract class Personaje {
  String nombre;

  Personaje({required this.nombre});
  void atacar();
  @override
  String toString() {
    return nombre;
  }
}
```

- La clase `Personaje` es abstracta ya que no tiene mucho sentido construir instancias de `Personaje`. Las instancias que tienen sentido son las de sus subclases: `Knight` y `Witcher`.
- El método `atacar()` es virtual puro. No tiene sentido darle ninguna implementación en la superclase. Solo las subclases concretas saben como es su modo de ataque.

A continuación cada una de las subclases:

```
class Knight extends Personaje {
  Knight({required String nombre}): super(nombre: nombre);

  factory Knight.copiaDe({required Witcher witcher}) {
    return Knight(nombre: witcher.nombre);
  }

  @override
  void atacar() {
    print('Realizando un: super powerful attack');
  }

  @override
  String toString() {
    return 'Soy: ${super.toString()}, knight';
  }
}
```

```
class Witcher extends Personaje {
  Witcher({required String nombre}): super(nombre: nombre);

  factory Witcher.copiaDe({required Knight knight}) {
    return Witcher(nombre: knight.nombre);
  }

  @override
  void atacar() {
    print('Realizando un: Magic attack');
  }

  @override
  String toString() {
    return 'Soy: ${super.toString()}, witcher';
  }
}
```

- Uno de los requisitos del juego es que en un momento dado, los personajes puedan cambiar su rol. Por ejemplo, un `Witcher` puede convertirse en `Knight` y viceversa. Por eso se ha añadido un **constructor de copia**, que como sabemos, en Dart se implementa mediante un constructor factoría.
- Cada subclase ha redeclarado el método `atacar()` según su modo de ataque.

Podemos probar esta primera implementación definiendo algunos personajes concretos y haciéndolos evolucionar. Por ejemplo el siguiente fragmento de código:

```
var witcher = Witcher(nombre: 'Geralt de Rivia');
print(witcher);
witcher.atacar();

var knight = Knight.copiaDe(witcher: witcher);
print(knight);
knight.atacar();
```

muestra por la consola:

```
Soy: Geralt de Rivia, witcher
Realizando un: Magic attack
Soy: Geralt de Rivia, knight
Realizando un: super powerful attack
```

El código hace lo que debe y se ha programado usando programación orientada a objetos. Sin embargo, para estar seguros de que es código de calidad, hace falta ver cómo de bueno es **aceptando los cambios**.

Tras la versión v1.0 ha llegado el momento de lanzar la versión 2.0. Como novedad para esta versión se va a incorporar un nuevo tipo de personaje: `Sorcerer` que también realizará un `magic attack`

¿Cómo de bien preparado está nuestro código para aceptar ese cambio?

Veamos. En primer lugar habrá que definir una subclase de `Personaje` llamada `Sorcerer`. Hasta ahí todo bien, sin embargo, hay varias cosas donde la estructura de clases actual no funciona bien:

1. Recordemos que uno de los requisitos del juego es que los personajes deben poder evolucionar y transformarse en otros personajes. Por eso, tras añadir la subclase `Sorcerer` hay que modificar las subclases ya existentes para añadirles un nuevo constructor de copia, el que produce una instancia de esas subclases a partir de una de la clase `Sorcerer`.

Modificar clases existentes para añadir nuevas funcionalidades viola el principio abierto-cerrado, OCP. **Nuestro código actual no es bueno.**

2. El nuevo personaje usa el mismo tipo de ataque de un personaje ya existente. No podemos duplicar el código del método `atacar()` de la clase `Witcher` en la clase `Sorcerer`, ya que estaríamos violando el principio de no te repitas, DRY. Menos aún hacer que `Sorcerer` sea subclase de `Witcher` porque por polimorfismo, ambas clases serían intercambiables.

Hemos llegado a un callejón sin salida. Un cambio sencillo en nuestro juego, añadir un nuevo tipo de personaje, obliga a modificar las clases existentes y a duplicar código. Nuestra aplicación no se adapta bien a los cambios por lo que **no es una buena aplicación.**

Hay que refactorizarla para mejorar la estructura del código y después ya podremos añadir el nuevo tipo de personaje.

Antes de empezar, es necesario averiguar dónde está el problema de fondo. Sin ese paso previo, probablemente acabemos en otro callejón sin salida.

1. Uno de los problemas es la necesidad de transformar unas subclases de personaje en otras mediante un constructor de copia. Eso complica añadir nuevos personajes, ya que obliga a modificar las clases de todos los personajes ya añadidos.

El fallo aquí es que no hemos entendido que transformar un personaje en otro, en el fondo es cambiar la forma en que ataca por otra. Es decir, no nos hemos dado cuenta de que lo que forma una jerarquía de clases no son los personajes en sí, sino las formas de atacar. Se trata de una familia de algoritmos encapsulados e intercambiables. Eso se programa mucho mejor usando el **patrón estrategia**.

2. El otro problema es que hemos codificado el tipo de personaje directamente en una clase, cuando en realidad se podría haber codificado mejor en **una propiedad de una clase**.

El fallo aquí es no haber entendido que `Sorcerer`, `Witcher` y `Knight` no son objetos de interés para el problema, sino casos particulares del objeto de interés que es `Personaje`. Eso nos ha llevado a identificar el tipo de personaje de que se trata mediante una subclase, en lugar de hacerlo mediante un tipo enumerado que se usa como propiedad de la clase `Personaje`.

Una vez identificados los problemas estamos en condiciones de refactorizar nuestro código para que use el patrón estrategia junto a tipos enumerados y a continuación añadir el nuevo tipo de personaje.

Pero antes necesitamos conocer qué es el patrón estrategia y cómo se usa.

En el patrón estrategia, hay tres actores principales: Una **estrategia**, varias **estrategias concretas** y un **contexto**.

1. La **estrategia** es una interface que define la API común que van a tener las estrategias concretas. Define la familia de algoritmos que se quiere encapsular.

En nuestro ejemplo, la estrategia es `ModoAtaque`. Encapsula todos los métodos de ataque que se requiera en el juego y los hace intercambiables, de forma que cambiar la forma en que un personaje ataca sea trivial. La API común de la estrategia será el método `atacar()`

Se define como una interface. En Dart no hay interfaces como tales, pero se pueden construir mediante clases abstractas con métodos virtuales puros. En este ejemplo, la estrategia se declararía así:

```
abstract class ModoAtaque {  
    void atacar();  
}
```

2. Las estrategias concretas son clases que implementan la interface estrategia. Definen los distintos algoritmos encapsulados en la estrategia.

En nuestro ejemplo, tendríamos dos: `SuperPowerfulAttack` y `MagicAttack` que se definen así:

```
class SuperPowerfulAttack implements ModoAtaque {  
    const SuperPowerfulAttack();  
    @override  
    void atacar() {  
        print('Realizando un: super powerful attack');  
    }  
}
```

```
class MagicAttack implements ModoAtaque {  
    const MagicAttack();  
    @override  
    void atacar() {  
        print('Realizando un: Magic attack');  
    }  
}
```

- Como podemos ver, cada estrategia concreta implementa su versión específica del método `atacar()`. Junto a la interface `ModoAtaque` definen una familia de algoritmos que encapsulan los distintos modos de ataque, y que por medio del **polimorfismo**, son **intercambiables**.

Si se define una propiedad de clase `ModoAtaque`, en un momento concreto de la ejecución de la aplicación podrá contener una instancia de `MagicAttack` o bien de `SuperPowerfulAttack` según convenga.

- Se ha añadido un **constructor constante** a cada clase. Eso tiene varias ventajas.
 - Podemos asociar una estrategia concreta a los valores del tipo enumerado (que son constantes). Lo veremos un poco más abajo.
 - Cuando se construyen estrategias concretas, se construyen **instancias canónicas**, lo que optimiza el uso de recursos por parte de la aplicación.
3. El último actor en el patrón estrategia es el contexto. Es la clase que usa la familia de algoritmos definida por la estrategia. En este caso, el contexto es la clase `Personaje` que debe refactorizarse.

- El primer paso es eliminar la palabra reservada `abstract` de la declaración de la clase `Personaje`. Esta clase ya no es una clase abstracta, sino que es una clase que puede instanciarse.
- A continuación hay que declarar en la clase `Personaje` una nueva propiedad de tipo `ModoAtaque` llamada `modoAtaque`.

```
ModoAtaque modoAtaque;
```

- Hay que inicializar la nueva propiedad en el constructor. Aquí se ha optado por usar una **lista de inicialización**, de forma, que cuando se crea una nueva instancia de la clase `Personaje` por defecto tenga como estrategia concreta asignada una instancia de `SuperPowerfulAttack`. No es la única forma de hacerlo.

Hay que sustituir el constructor de la clase:

```
Personaje({required this.nombre});
```

por este otro:

```
Personaje({required this.nombre}):  
    modoAtaque = SuperPowerfulAttack();
```

- Por último hay que dotar de cuerpo al método `atacar()` de la clase `Personaje` que ya no es un método virtual puro.

Cuando se invoque a este método, debe **delegar**, en el método `atacar()` de su estrategia concreta. Sustituimos:

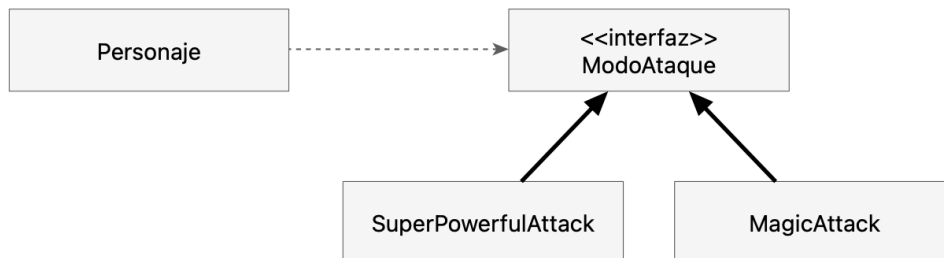
```
void atacar();
```

por:

```
void atacar() {  
    modoAtaque.atacar();  
};
```

Cuando en una instancia de la clase `Personaje` se tenga como `modoAtaque` una instancia de la clase `MagicAttack` y se invoque el método `atacar()`, por el polimorfismo, el código que se ejecutará será el definido en la clase `MagicAttack`. Del mismo modo, si lo que hay en `modoAtaque` es una instancia de `SuperPowerfulAttack`, el código que se ejecutará será el del método `atacar()` de esta clase.

Ya tenemos implementado el patrón estrategia para definir una familia de algoritmos, los modos de ataque, de forma que cada modo esté encapsulado en su propia clase y sean intercambiables.



Ahora es el momento de definir un tipo enumerado, para que los cambios de tipo de personaje no obliguen a modificar el código existente. Recordemos que en este tipo enumerado vamos a codificar el tipo de personaje de que se trate, así que añadiremos el siguiente tipo:

```
enum TipoPersonaje {
    knight('Knight', SuperPowerfulAttack()),
    witcher('Witcher', MagicAttack());

    const TipoPersonaje(this.nombre, this.modaAtaque);

    final String nombre;
    final ModoAtaque modaAtaque;
}
```

- Como vemos, se definen dos constantes en el tipo enumerado, `knight` y `witcher`, una por cada tipo de personaje.
- A cada una se le asocian dos propiedades: el `nombre` asociado a ese tipo de personaje y el `modaAtaque` asociado a ese personaje.

Ya solo queda usar ese tipo enumerado en la clase `Personaje`. Para ello, le añadiremos la siguiente propiedad:

```
TipoPersonaje _tipoPersonaje;

set tipoPersonaje(TipoPersonaje nuevoTipo) {
    _tipoPersonaje = nuevoTipo;
    modaAtaque = _tipoPersonaje.modaAtaque;
}
```

- Como vemos, se ha definido un `setter` de forma que cuando se cambia el tipo de personaje en la clase `Personaje` se cambia también automáticamente el modo de ataque que ese tipo lleva asociado.

El método `toString` de la clase `Personaje` por ahora, solo devuelve el `nombre` del personaje, ya que el tipo de personaje se añadía en la redeclaración en las subclases. Para obtener un mensaje en consola más claro, podemos cambiarlo por:

```
@override
String toString() {
    return 'Soy $nombre, un ${_tipoPersonaje.nombre}';
}
```

Con todo esto ya tenemos refactorizado el código para que añadir un nuevo personaje al juego sea muy sencillo. Antes de hacerlo vamos a probarlo.

En el `main` escribimos:

```
var personaje = Personaje(nombre: 'Geralt de Rivia');
print(personaje);
personaje.atacar();

personaje.tipoPersonaje = TipoPersonaje.witcher;

print(personaje);
personaje.atacar();
```

lo que produce la siguiente salida en la consola:

```
Soy Geralt de Rivia, un Knight
Realizando un: super powerful attack
Soy Geralt de Rivia, un Witcher
Realizando un: Magic attack
```

Ahora es el momento de añadir el nuevo personaje `sorcerer` con tipo de ataque `MagicAttack`. Para ello, solo hay que hacer una cosa:

1. Sustituir la línea del tipo enumerado `TipoPersonaje` :

```
witcher('Witcher', MagicAttack());
```

por las líneas:

```
witcher('Witcher', MagicAttack()),
sorcerer('Sorcerer', MagicAttack());
```

Eso es todo. Mucho más sencillo que en la primera versión del código. Para probarlo, podemos añadir al código de ejemplo anterior:

```
personaje.tipoPersonaje = TipoPersonaje.sorcerer;

print(personaje);
personaje.atacar();
```

que añade a la salida en consola:

```
Soy Geralt de Rivia, un Sorcerer  
Realizando un: Magic attack
```

Retorno al pasado, el patrón orden (command)

El patrón orden (command) es la solución al siguiente problema:

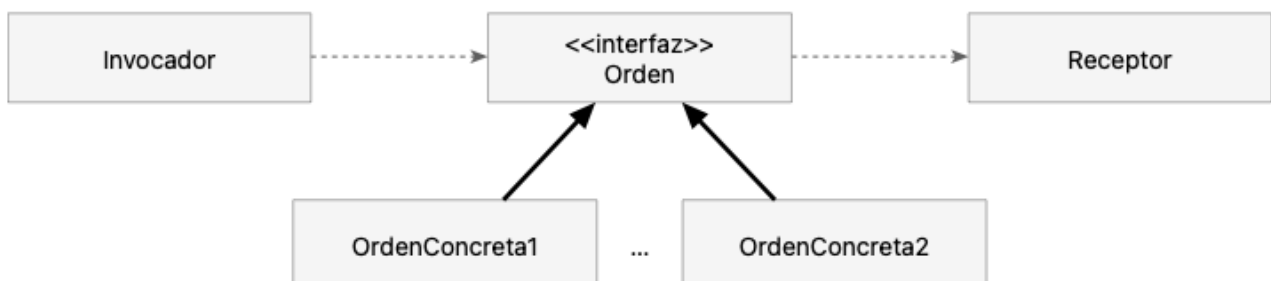
- **Encapsular** las tareas que ejecuta la aplicación dentro de una **familia de clases**, de forma que todas las tareas puedan ser:
 - tratadas de la misma forma aunque sean tareas muy diferentes.
 - almacenadas en listas de tareas que puedan ejecutarse en orden de forma automática.
 - deshechas para anular su efecto sobre el estado de la aplicación.

La única forma, adaptada a los cambios, de dotar a nuestras aplicaciones móviles de la capacidad de deshacer tareas que puedan cambiar el estado de la aplicación de forma no deseada, es aplicando el patrón orden. Este patrón asegura que en el futuro se podrán añadir nuevas tareas con la capacidad de deshacer sin tener que modificar nada de lo ya programado en la aplicación.

Nota: Los aspectos clave en este patrón son **encapsular** y **familia de clases**. Se trata de que todas las órdenes de una aplicación estén encapsuladas en una familia de clases de forma que todas ellas se traten igual. Unas órdenes serán muy complejas, mientras que otras serán muy sencillas. Unas órdenes serán ejecutadas por unas clases de la aplicación mientras que otras serán ejecutadas por otras clases. Sin embargo, todas ellas deben tratarse de la misma forma.

Veamos un ejemplo sencillo. Imagina una aplicación móvil, en la que hay una clase del modelo de negocio de la aplicación, que contiene una lista de enteros. Una de las acciones que se puede realizar sobre esa lista es borrarla y queremos poder deshacer esa acción.

Para lograr eso, en el patrón orden (command) hay cuatro actores principales: **invocador** de la orden, **receptor** de la orden, una **orden** y varias **órdenes concretas**. El diagrama de clases es así:



1. La **orden** es una interface que **define la API común** que van a implementar todas las órdenes concretas. En concreto define tres métodos: `execute`, `undo` y `redo`, los tres sin parámetros.

```
abstract class Orden {  
    void execute();  
    void undo();  
    void redo();  
}
```

2. Normalmente, el patrón orden se usa junto con un `UndoManager` que no es sino dos pilas de órdenes. La pila de órdenes pendiente de deshacerse y la pila de órdenes pendientes de rehacerse. Se usan pilas, ya que las órdenes se deshacen en orden contrario al orden en que se han ejecutado. Hace falta una pila (estructura de tipo LIFO).

Nota: La clase `Pila` que se usa aquí es la clase `Stack` que has implementado en el ejercicio 5 del bloque 8 de Dart. Copia esa clase y renómbrala en este proyecto.

El `UndoManager` se suele implementar como un **Singleton**, ya que solo debe haber uno para toda la aplicación donde se almacenen todas las órdenes ejecutadas y pendientes de deshacer, así como las que se hayan deshecho y estén pendientes de rehacerse.

```
class UndoManager {  
    final _undoPila = Pila<Orden>();  
    final _redoPila = Pila<Orden>();  
  
    UndoManager._();  
  
    static final UndoManager instance = UndoManager._();  
  
    void add({required Orden orden}) {  
        _undoPila.push(orden);  
        _redoPila.clear();  
    }  
  
    void undo() {  
        if (!_undoPila.isEmpty) {  
            final command2Undo = _undoPila.pop();  
            command2Undo.undo();  
            _redoPila.push(command2Undo);  
        }  
    }  
  
    void redo() {  
        if (!_redoPila.isEmpty) {  
            final command2Redo = _redoPila.pop();  
            command2Redo.redo();  
            _undoPila.push(command2Redo);  
        }  
    }  
  
    @override  
    String toString() {  
        return 'Commands to undo: $_undoPila\nCommands to redo: $_redoPila';  
    }  
}
```

- Un `UndoManager` necesita un método `add` para añadir las nuevas órdenes que se hayan ejecutado. Este método añade la orden a la pila de órdenes para deshacer y borrar el contenido de la pila de órdenes para rehacer.
Si no se hace esto último, la cadena de órdenes se vería alterada y el programa no se ejecutaría del modo esperado.
- También necesita el método `undo`. La tarea de este método es comprobar si hay órdenes en la pila de órdenes para deshacer. Si las hay saca la primera (la última que se añadió a esta pila), llama al método `undo` de esa orden para que se deshaga la tarea correspondiente y a continuación la añade en la cabecera de la pila de órdenes para rehacer. De esta forma estará preparada por si el usuario se arrepiente y quiere volver a rehacer la orden.
- Por último, también necesita el método `redo`, que comprueba si la pila de órdenes para rehacer está vacía, si no lo está, saca la primera (la última que se colocó en esta pila), llama al método `redo` de esa orden para que se vuelva a rehacer la tarea correspondiente y la añade en la cabecera de la pila de órdenes para deshacer.

3. El **receptor** de la orden es quien realiza la tarea. Normalmente es quien contiene el estado que va a ser cambiado por la orden y que se desea que se pueda restaurar a su valor anterior al deshacer la orden. En este ejemplo, la clase `Receptor` contiene la lista de enteros.

```
class Receptor {
    var _lista = <int>[];

    List<int> get lista => _lista.toList();
    set lista(List<int> nuevoValor) => _lista = nuevoValor;

    void borrarLista() {
        _lista.clear();
    }

    @override
    String toString() {
        return _lista.toString();
    }
}
```

- Además de la lista de enteros, se ha definido el método `borrarLista()` que es el que realiza la tarea que cambia el estado de la aplicación.
- Además se ha redeclarado el método `toString` para que sea sencillo imprimir el contenido en la consola al hacer las pruebas.
- También se ha declarado un getter y un setter para la lista, de forma que la orden pueda guardar el estado anterior y de esa forma pueda restaurarlo al deshacer.
Fíjate que el getter `lista` devuelve una **copia profunda** de la lista almacenada. Si no se hace así, lo que se devuelve es una referencia, con lo que la orden no estaría almacenando el estado anterior, sino el actual y no serviría para deshacer.

4. Lo siguiente es declarar una **orden concreta** para encapsular el proceso de borrar la lista que está contenida en la clase `Receptor`

```
class OrdenBorrarLista implements Orden {
    Receptor receptor;
    var estadoAnterior = <int>[];

    OrdenBorrarLista(this.receptor);

    @override
    void execute() {
        estadoAnterior = receptor.lista;
        receptor.borrarLista();
        UndoManager.instance.add(orden: this);
    }

    @override
    void undo() {
        receptor.lista = estadoAnterior;
    }

    @override
    void redo() {
        receptor.borrarLista();
    }
}
```

- La orden concreta `OrdenBorrarLista` implementa la interface `Orden`, por tanto **debe** redeclarar los métodos comunes a todas las órdenes: `execute`, `undo` y `redo` que son virtuales puros.
- Para poder operar necesita saber quien es el receptor con el que tiene que trabajar. Se le pasa como argumento en el constructor.
- Además, debe tener una propiedad en la que almacenar el estado anterior. En esta propiedad se guardará la lista tal y como está en el momento de ejecutar el borrado. De esta forma, si se deshace la orden, se puede recuperar el valor guardado y volver a asignarlo a la lista.
- El método `execute()` debe hacer tres cosas:
 - guardar una copia del estado actual de la lista de enteros. De esta forma, si se deshace el borrado se tiene esta copia para poder restaurarla en la clase `Receptor`
 - a continuación ejecutar el método `borrarLista` del receptor. Recuerda que la orden no es quien realiza la tarea, el borrado lo hace el receptor, que es quien posee el estado que se quiere cambiar. La orden lo que hace es **pedirle al receptor que realice la tarea**.
 - por último un paso fundamental. **La propia orden se añade a sí misma al `UndoManager`**. Nadie mejor que una orden para saber cuando se ha ejecutado, por eso, al final del método `execute` la orden debe añadirse al `UndoManager`.

- El método `undo()` es el encargado de colocar en el receptor, la copia de la lista, tal y como estaba en el momento en que se ejecutó el borrado. De esta forma se deshace el efecto del borrado.
- El método `redo()` se encarga de volver a borrar la lista, para dejar el estado de la aplicación tal y como quedó tras la ejecución de la orden. En este caso, no hay que recordar nada solo volver a ejecutar el borrado de la lista de enteros.

Importante: Cada orden concreta encapsula **una tarea** de las que se realizan en la aplicación. La orden `OrdenBorrarLista` se encarga de encapsular el proceso de borrar la lista de enteros y preparar todo lo necesario para poder deshacer ese proceso. Si se necesita implementar más tareas que modifiquen el estado de la aplicación, por ejemplo, una tarea para añadir elementos a la lista, se debe implementar **otra orden** concreta para encapsular esa nueva tarea.

5. En este ejemplo sencillo, no se ha programado un invocador. En una aplicación móvil real, el invocador es una parte de la interfaz de usuario de la aplicación. Por ejemplo, un botón que desencadena una acción.

Aquí usaremos el método `main` como invocador y nos permitirá probar el patrón. Si escribimos el siguiente fragmento de código en el `main` :

```
Receptor receptor = Receptor();

receptor.lista = [1, 2, 3, 4, 5];
print(receptor);

OrdenBorrarLista(receptor).execute();
print(receptor);
print(UndoManager.instance);

UndoManager.instance.undo();
print(receptor);
print(UndoManager.instance);

UndoManager.instance.redo();
print(receptor);
print(UndoManager.instance);
```

y lo ejecutamos, se obtiene en la consola:

```
[1, 2, 3, 4, 5]
[]
Commands to undo: [Instance of 'OrdenBorrarLista']
Commands to redo: []
[1, 2, 3, 4, 5]
Commands to undo: []
Commands to redo: [Instance of 'OrdenBorrarLista']
[]
Commands to undo: [Instance of 'OrdenBorrarLista']
Commands to redo: []
```

Como podemos ver como, tanto la lista de enteros, como la pila de órdenes para deshacer/rehacer evolucionan tal y como queremos.

Mini ejercicios

1. ➡ Añade un nuevo pájaro no volador: el pollito, a la **versión final del juego** que se ha estudiado en la sección dedicada al principio de sustitución de Liskov. ¿Cómo de difícil ha sido añadirlo? ¿Dónde has tenido que añadir código? ¿Dónde has tenido que modificar el código usuario de la jerarquía de clases cuyo nodo raíz es `Pajaro`?
2. ➡ Escribe un Singleton para gestionar **ids únicos** para objetos, de forma que cuando algún constructor de otra clase necesite un id de tipo `int` el Singleton le asigne un valor que aún no se haya usado en ninguna otra instancia de ninguna otra clase.

Escribe también un pequeño ejemplo que ilustre cómo se usa.

Explica las razones por la que crees que es bueno usar un Singleton para esta tarea.

3. ➡ Añade un nuevo modo de ataque al juego llamado **mystic attack** y asígnalo al tipo de personaje llamado `sorcerer`. Explica qué has tenido que hacer. ¿Se ha violado algún principio de diseño de software? Explica detalladamente tu respuesta.
4. ➡ ¿Y si la clase `Personaje` de este capítulo, además de tener un modo de ataque asociado tuviese un modo de moverse también intercambiable? ¿Cómo lo programarías?
5. ➡ Usa la respuesta del ejercicio anterior para añadir a la clase `Personaje` la capacidad de movimiento siguiente:

`Knight` tiene un movimiento corto.
`Witcher` tiene un movimiento corto.
`Sorcerer` tiene un movimiento largo.

Explica qué has tenido que hacer. ¿Se ha violado algún principio de diseño de software?

6. En el patrón estrategia, se usa el mecanismo conocido como **delegación** ¿En qué consiste? ¿Cuál de los papeles o roles que intervienen en el patrón estrategia es quien delega parte de su funcionalidad? ¿En qué otro papel o rol la delega?
7. ➡ Refactoriza la aplicación para dibujar pájaros de la sección dedicada al principio de sustitución de Liskov, de forma que no se viole ese principio y la solución sea mucho más flexible.

Una vez refactorizada, añade dos nuevos pájaros, el `Pollo` que no vuela y la `Paloma` que vuela en un rango de entre 0 y 250. ¿Cómo de fácil ha sido añadir estas clases?

Pista: Usa el patrón estrategia y los tipos enumerados.

8. ¿Qué tareas de una aplicación móvil deberían poder deshacerse? Explica detalladamente tu respuesta.

9. ➡ Añade una nueva tarea a la aplicación que ilustra el apartado dedicado al patrón orden. Esa tarea debe permitir añadir un nuevo entero a la lista por el final.

¿Cómo lo has hecho? Explica detalladamente los pasos que hayas dado y justifica por qué son necesarios.